



National Teachers College

629 J Nepomuceno, Quiapo, Manila, 1001 Metro Manila
Bachelor of Science in Information Technology

ResQNet PH: A Real-Time Flood Monitoring and Community Alert System Using Crowdsourced Data and Predictive Analytics

Submitted by :

Co, Brix Aaron
Domandan, Louisse Janiel B.
Doromal, Ryan C.
Padilla, Allyza B.
Tan, Jan Marckey A.

BSIT 1.3

Instructor:

Neypes, Justin Louise R.

Date:

May 17, 2026

Final Project/Exam: C++ Solution for Sustainable Development

Section 1: 1.1 ResQNet PH is a C++ console tool that helps connect weather updates to what is really happening in local communities.

1.2 Why Do We Need This Software?

- The Philippines has weather agencies like PAGASA. Still when floods hit areas there is a lack of information. ResQNet PH was created because of three issues:
- **Missing Local Details:** National reports are too general. They do not provide information that residents need to stay safe. For example they may say it is raining in the city. They do not tell people if the river near their street is overflowing.
- **Rumors and Misinformation:** Without a local app people turn to social media for updates. This leads to rumors, confusion and panic.
- **Acting Late:** Without a central system local rescue teams usually react after the flood has happened. They make decisions too late putting lives and property at risk

Section 2: Requirements Analysis

2.1 Functional Requirements (FR): What the system *does* (e.g., "FR1: System must calculate carbon footprint").

FR 1: Flood Risk Assessment

The system needs to analyze and collected data, specifically water levels, to automatically classify areas into three levels. Low,Moderate, or High Risk.

FR 2: Data Input Handling

The system must be capable of accepting information entered by the user

FR 3: Safe Route Identification

The system will provide alternative routes that are safe and passable for people based on real-time flood conditions

FR 4: Alert Generation

The application will immediately issue a warning or emergency alert when a specific area enters - High Risk - status

FR 5: Data processing and Output

The system will process all the input details and present them in a clean and easy-to-read format for end-users

2.2 Non-Functional Requirements (NFR): Performance, Robustness (Input Validation), and Readability.

- **Performance**

The application needs to load and display data quickly without noticeable delays. Since this is used for flood monitoring, fast response times and real-time updates are critical for user safety.

- **Accuracy**

The risk classification logic must be spot-on and dependable. Because inaccurate data can put lives at risk, the algorithm for sorting areas into Low, Moderate, or High risk needs to be highly reliable..

- **Usability**

The user interface will be kept straightforward and uncluttered.

- **Robustness**

Proper input validation will be implemented to prevent the app from breaking.

- **Readability and maintainability**

The source code must be kept clean, well structured and properly commented.

- **Section 3: Design Specification**

3.1 **Algorithmic Flowchart:** A detailed flowchart of the system's most complex logic.

START

|

Initialize ResQNet System

(FloodMonitor monitor — displayHeader())

|

Load Existing Zone Data (Auto on Startup)

loadFromFile("database.txt") — runs before menu appears

Skips silently if file not found

|

Display Main Menu (Switch Case Driver)

[1] Input / Add Flood Zone Report

[2] Generate Predictive Risk Assessment

[3] View Safe Route Suggestions

[4] CRUD Zone Management

[5] Search Zone by ID

[6] Search Zone by Name

[7] Statistics Dashboard

[0] Save & Exit

|

Select Option (0–7)

|

[If Option 1 Selected] → logFloodReports()

|

Input Zone Information:

- Zone Name (Barangay)
- Municipality
- Water Level (cm) — validated via cin loop
- Number of Families Affected
- Evacuation Status (y/n)
- Rescue Team Assigned

|

Validate Input

|

Is data valid? (isValidLevel — 0 to 500 cm)

| — NO → [!] Invalid. Must be 0–500 cm → Return to Input

| — YES

|

Compute Risk Indicators

generateNewId() — assign unique zone ID

|

Check Critical Condition:

Is Water Level ≥ 200 OR Families ≥ 600 ?

| — YES

| |

| Set Alert Level = RED

| Set Priority = HIGH

| Create CriticalZone object

| Recommend Immediate Evacuation

|

└─ NO

|

Check Low Risk:

Is Water Level < 50 AND Families < 200?

└─ YES

| |

| Set Alert Level = GREEN

| Set Priority = LOW

| Create MonitoredZone object

| No Evacuation Needed

|

└─ NO

|

Set Alert Level = ORANGE

Set Priority = MEDIUM

Create FloodZone (base) object

Prepare Evacuation Plan

|

Check Evacuation Status

|

Is Evacuation = YES?

└─ YES → Assign Rescue Team & Mark as "In Progress"

└─ NO → Zone not yet evacuated — Monitor Continuously

|

Update RESQNET Database / Zone Record

zones[] vector — push new FloodZone* pointer

|

Save Data to File (Persistent Storage)

saveToFile() — write all zones to database.txt

|

Display Output:

- Zone Info (ID, Name, Municipality)
- Alert Level (RED / ORANGE / GREEN)
- Priority Level (HIGH / MEDIUM / LOW)
- Rescue Team & Evacuation Status
- z->printDetails() — polymorphic call

|

Return to Main Menu / Process Another Zone?

└─ YES → Go back to Switch Case Driver / Input

└─ NO

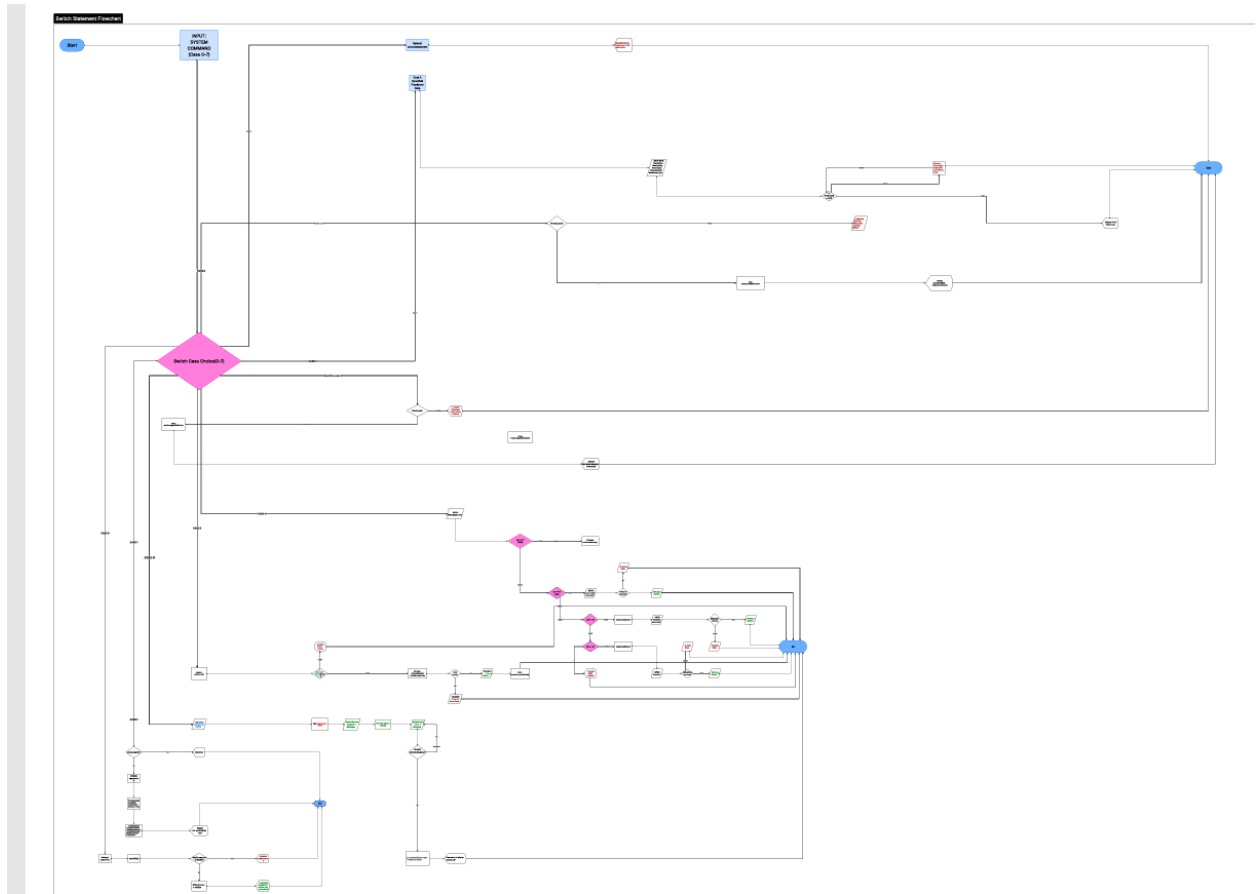
|

Terminate System Session

exitAndSave() — final save + goodbye message

|

END



3.2 File Structure Diagram: Explanation of the data format used in external files.

The system uses a structured file format to store and manage flood monitoring data efficiently. All zone records are saved in an external file “Zones.txt.” using a table-like structure where each row represents a single zone and each column represents a specific data field

ZoneID	ZoneName	Municipality	WaterLevel	Families	Evacuated	AlertLevel
--------	----------	--------------	------------	----------	-----------	------------

Z001	Riverside	Manila	210	650	Yes	RED
------	-----------	--------	-----	-----	-----	-----

Z002	Poblacion	Quezon	120	300	No	ORANGE
------	-----------	--------	-----	-----	----	--------

Z003	San Isidro	Bulacan	40	150	No	GREEN
------	------------	---------	----	-----	----	-------

ZoneID – Unique identifier for each area

ZoneName – Name of the zone or location

Municipality – City or province where the zone is located

WaterLevel – Measured water level (in centimeters)

Families – Number of affected families

Evacuated – Indicates whether evacuation has started (Yes/No)

AlertLevel – System generated risk level (GREEN, ORANGE, RED)

This piece of C++ code is an example of an excellent practical application of the Object-Oriented Programming (OOP) concept for designing a flood monitoring system to be used in cases of disasters and floods. The structure of this application is done in a way that makes it highly organized and scalable. It takes advantage of the principles behind OOP, including Encapsulation, Abstraction, Inheritance, and Polymorphism.

Encapsulation: Protecting the Code from External Modification

One of the basic ways of building a good flood monitoring program is ensuring security and protecting data from external sources. Encapsulation is one of the principles of object-oriented programming and can be implemented in the provided code through the private access modifier. It means that variables such as floodLevel, affectedFamilies, and rescueTeam can no longer be modified outside the code since their direct access is limited. The programmer would not be able to set the flood level as being negative or change the name of the municipality using the provided code because it would simply not allow it.

Abstraction: Concealing System Complexity

Although Encapsulation guards the data, Abstraction simplifies its access. For instance, one can see the use of Abstraction in FloodZone::getRiskStatus(). To calculate the risk status, a complex computation engine evaluates the waterScore, which is dependent on the depth of the flood, and popScore, which depends on the number of families affected. The resulting combination of waterScore and popScore is then used to compute the threat level. Calling the function is as simple as writing myZone.getRiskStatus() "[CRITICAL -

EVACUATE NOW]". With the help of Abstraction, programmers can leverage FloodZone objects without worrying about their complex mathematical computations.

Inheritance: Scaling Hierarchies

Because the disaster management process automatically splits into different scenarios, Inheritance is employed to eliminate redundant code. In this case, the base class, FloodZone, defines the template that includes all common attributes and methods of every flooded region. Whenever there is a need for customization, derived classes are created, called CriticalZone and MonitoredZone. With the help of class CriticalZone : public FloodZone syntax, each child class inherits all attributes and methods of the base class. On the other hand, children only implement the new features unique to their specialization, such as the alertLevel string variable in CriticalZone

Polymorphism: Dynamic and Contextual Behavior

It is here, in polymorphism, that the real potential of this architecture is unleashed by making objects from multiple classes appear like objects of a single base class yet allowing them to retain their characteristic behaviors. The methods getActionPlan() and printDetails() have been designated by the keyword virtual in FloodZone.h. This tells the compiler that it can be overridden in any number of subclasses. By specifying the override keyword for getActionPlan() in the derived classes CriticalZone and MonitoredZone, we effectively replace the general advice of the parent class with very context-specific recommendations.

CriticalZone would immediately call for mass evacuation while MonitoredZone would suggest a more passive stance of watch-and-wait. With the help of polymorphism, a central command center application could iterate an array of generic pointers to FloodZones. Although they would be seen as merely simple flood zones, invoking the method getActionPlan() on each one of them would automatically execute its overridden version depending on whether the particular object belongs to the CriticalZone or MonitoredZone subclass.

Contributors:

Tan, Jan Marckey A. developed the FloodMonitor.cpp file, including the private attributes, constructors, getters and setters, getRiskStatus(), getUrgencyScore(), and toFileString() methods. He also created the FloodMonitor.h header file and managed the GitHub repository, ensuring proper organization and file management throughout the project.

Padilla, Allyza B. handled the derived classes CriticalZone and MonitoredZone. She implemented their header and source files, constructors, overridden getActionPlan() methods, and overridden printDetails() methods. She also prepared the database.txt file containing 20 Metro Manila barangay records, conducted program testing, and organized the presentation slides for the defense.

Co, Brix Aaron focused on the file handling functionalities in FloodMonitor.cpp. He implemented the constructor, destructor, helper methods generateNewId() and findIndexById(), as well as the loadFromFile() and saveToFile() methods for data persistence. He also prepared the README.md and organized the project folder structure in GitHub.

Doromal, Ryan C. was responsible for the reporting and analytics features of the system. He implemented the methods displayHeader(), displayMenu(), logFloodReports(), calculateRiskStats(), generateRiskAssessment(), suggestSafeRoutes(), and showStatsDashboard(), which support flood-related analysis and decision-making. In addition, he compiled and submitted the SDAD documentation.

Domandan, Louisse Janiel B. implemented the remaining FloodZone methods, including addZone(), viewZones(), updateZone(), deleteZone(), searchById(), searchByName(), isDataLoaded(), isValidLevel(), and exitAndSave(). She also developed the main.cpp file, including the program's main loop and switch statement, and created the project flowchart.